# Réplica Skill

## 4 apps para clonar con Claude Code

Cómo instalarlo, los prompts exactos y el paso a paso
para clonar cada app desde cero.

01 Link in bio

02 Agendado de citas

03 Formularios y encuestas

04 CRM ligero

@ortegoat

EL REPO EN UNA HOJA

# Qué es Réplica Skill

Son 11 skills gratuitas para Claude que trabajan en cadena para clonar las **funciones** de una app: la analizan, la reconstruyen, la prueban, la comparan con la original y te ayudan a lanzarla con tu propia marca. Cada skill guarda su trabajo en una carpeta **replica/** y la siguiente lo lee. Licencia MIT, sin registro y sin API key.

## Las 11 skills

| | | | |
|---|---|---|---|
| **/replica-recon** | Analiza la app original | **/replica-architect** | Planea stack y base de datos |
| **/replica-design** | Reconstruye el diseño | **/replica-build** | Construye pantalla por pantalla |
| **/replica-backend** | Login, pagos, integraciones | **/replica-test** | Busca errores |
| **/replica-diff** | Puntaje de parecido y lo que falta | **/replica-entrepreneur** | Lee reseñas y propone mejoras |
| **/replica-brand** | Nombre y marca propia | **/replica-launch** | Landing, precios y ficha de tienda |
| **/replica-deploy** | Publica en tu dominio | | |

## Instalación en Claude Code

Desde tu terminal, copia las skills a tu carpeta global de Claude Code. Así quedan disponibles como **/replica-recon**, **/replica-build**, etc., en cualquier proyecto.

```
git clone https://github.com/Jakeschincariol/replica-skill.git
cp -r replica-skill/replica-* ~/.claude/skills/
```

Otra opción: dentro de Claude Code o en el chat de Claude, pega el link del repo seguido de **install skill**. Requisito: Python 3.8 o más reciente (no hay que instalar nada con pip).

## Una regla antes de empezar

La herramienta copia **lo que una app hace**, no lo que le pertenece: nunca su código, logo, textos ni contenido. Antes de publicar, /replica-brand te obliga a cambiar nombre y marca, y /replica-deploy no publica hasta que no quede rastro de la original.

CASO DE USO 01

# 01 Link in bio

Tipo Linktree

| DIFICULTAD | IDEAL PARA | APP DE REFERENCIA |
|---|---|---|
| **Baja** | **Tu primer clon** | **Linktree** |

## Por qué conviene

Pocas pantallas, flujos simples y una app que cobra mensualidad por funciones como temas, estadísticas y quitar su marca. Es perfecta para aprender el flujo completo de las 11 skills antes de clonar algo más grande.

## Qué vas a construir

- Página pública de perfil con foto, bio y lista de links
- Editor para agregar, ordenar y ocultar links
- Temas visuales personalizables
- Estadísticas de clics por link

## Paso 0: abre tu proyecto en Claude Code

Desde tu terminal crea una carpeta para el clon y abre Claude Code dentro:

```
mkdir clon-linkinbio && cd clon-linkinbio
claude
```

## Prompts paso a paso

Copia cada prompt en Claude Code en este orden. Espera a que termine cada paso antes del siguiente.

### 1 Analizar la app

```
/replica-recon https://linktr.ee

Analiza esta app de link in bio. Enfócate en el editor de links, la página
pública del perfil, los temas visuales y las estadísticas de clics. Marca como
fuera de alcance la tienda, los pagos a creadores y las integraciones con otras
plataformas.
```

*Obtienes replica/recon.md con pantallas y flujos, y features.csv con cada función.*

### 2 Planear la arquitectura

```
/replica-architect

Quiero una arquitectura simple y barata de mantener. Next.js, Postgres y
despliegue en Vercel. Prioriza que la página pública del perfil cargue muy
rápido.
```

## 3 Diseño

```
/replica-design

Usa estas capturas de la app original como referencia de estructura. Crea un sistema de diseño propio con una paleta neutra y 3 temas que el usuario pueda elegir.
```

*Tip: arrastra las capturas a Claude Code antes de enviar el prompt.*

## 4 Construir

```
/replica-build

Construye primero la página pública del perfil y el editor de links de punta a punta. Después los temas y al final las estadísticas.
```

## 5 Backend y pruebas

```
/replica-backend

Agrega registro con correo y Google, y guarda cada clic con fecha para las estadísticas. Después corre /replica-test.
```

## 6 Medir qué tan igual quedó

```
/replica-diff

Compara mi clon contra la original y dame el puntaje de paridad y la lista de funciones obligatorias que faltan.
```

*Si falta alguna función obligatoria, regresa a /replica-build con esa lista antes de seguir.*

## 7 Diferenciarte y lanzar

```
/replica-entrepreneur > /replica-brand > /replica-launch > /replica-deploy

En entrepreneur: busca quejas sobre precios y funciones bloqueadas detrás del plan de paga.
En brand: propón 5 nombres en español para creadores de LATAM.
```

**TIP PARA ESTE CASO**

Ángulo de venta: una versión en español con todas las funciones incluidas en un solo precio bajo, pensada para creadores de LATAM.

CASO DE USO 02

# 02 Agendado de citas

Tipo Calendly

| DIFICULTAD | IDEAL PARA | APP DE REFERENCIA |
|---|---|---|
| **Media** | **Negocios locales** | **Calendly** |

## Por qué conviene

Es el ejemplo oficial del repo y uno de los más útiles: barberías, consultorios, estudios y asesores necesitan que sus clientes reserven solos. Además, las herramientas grandes cobran por usuario, lo que encarece mucho a equipos pequeños.

## Qué vas a construir

- Link público donde el cliente elige día y hora
- Configuración de disponibilidad y tipos de cita
- Reprogramar y cancelar
- Recordatorios por correo y sincronización con Google Calendar

## Paso 0: abre tu proyecto en Claude Code

Desde tu terminal crea una carpeta para el clon y abre Claude Code dentro:

```
mkdir clon-agendado && cd clon-agendado
claude
```

## Prompts paso a paso

Copia cada prompt en Claude Code en este orden. Espera a que termine cada paso antes del siguiente.

### 1 Analizar la app

```
/replica-recon https://calendly.com

Analiza esta app de agendado. Quiero los flujos de: el cliente reserva una cita,
el dueño configura su disponibilidad y el cliente reprograma. Marca como fuera de
alcance el marketplace de integraciones y las funciones para empresas grandes.
```

*En el ejemplo del repo, este paso encontró 18 pantallas y 7 flujos.*

### 2 Planear la arquitectura

```
/replica-architect

Next.js, Postgres, Stripe para citas pagadas y Resend para correos. Agrega una
restricción en la base de datos para que dos clientes nunca puedan reservar el
mismo horario.
```

**3** **Diseño**

```
/replica-design

Toma estas capturas como referencia de estructura del calendario y los botones de horario. Crea una identidad propia, limpia y fácil de usar en celular.
```

**4** **Construir**

```
/replica-build

Construye primero el flujo de reserva completo de punta a punta. Después la configuración de disponibilidad y al final reprogramar y cancelar.
```

**5** **Backend y pruebas**

```
/replica-backend

Agrega login, conexión con Google Calendar con mis propias llaves, pagos con Stripe y recordatorios por correo. Después corre /replica-test y prueba especialmente zonas horarias y dos personas reservando el mismo horario al mismo tiempo.
```

**6** **Medir qué tan igual quedó**

```
/replica-diff

Compara mi clon contra la original. Dame el puntaje de funciones, el puntaje de diseño de la página de reservas y lo que falta.
```

*En el ejemplo del repo: 86 en funciones y 91 en diseño de la página de reservas.*

**7** **Diferenciarte y lanzar**

```
/replica-entrepreneur > /replica-brand > /replica-launch > /replica-deploy

En entrepreneur: busca quejas sobre precio por usuario, recordatorios y zonas horarias.
En launch: pon un precio fijo mensual sin cobro por usuario.
```

**TIP PARA ESTE CASO**

Agrega recordatorios por WhatsApp. En LATAM es mucho más útil que el correo y es una queja frecuente de las apps de agendado.

CASO DE USO 03

# 03 Formularios y encuestas

Tipo Typeform

| DIFICULTAD | IDEAL PARA | APP DE REFERENCIA |
|---|---|---|
| **Media** | **Creadores y agencias** | **Typeform** |

## Por qué conviene

Los planes de pago de estas herramientas suben rápido por funciones como lógica condicional, más respuestas o quitar su marca. Todo eso son funciones, no contenido, así que se pueden reconstruir completas.

## Qué vas a construir

- Editor de formularios con distintos tipos de pregunta
- Formulario público de una pregunta por pantalla
- Lógica condicional (saltar preguntas según la respuesta)
- Panel de respuestas con exportación a CSV

## Paso 0: abre tu proyecto en Claude Code

Desde tu terminal crea una carpeta para el clon y abre Claude Code dentro:

```
mkdir clon-formularios && cd clon-formularios
claude
```

## Prompts paso a paso

Copia cada prompt en Claude Code en este orden. Espera a que termine cada paso antes del siguiente.

### 1 Analizar la app

```
/replica-recon https://www.typeform.com

Analiza esta app de formularios. Enfócate en el editor, la experiencia de
responder una pregunta por pantalla, la lógica condicional y el panel de
respuestas. Marca como fuera de alcance las plantillas de su galería y su
marketplace de integraciones.
```

### 2 Planear la arquitectura

```
/replica-architect

Next.js y Postgres. Diseña el esquema para que los formularios, las preguntas y
las reglas de lógica sean flexibles y las respuestas se puedan exportar
fácilmente.
```

**3** **Diseño**

```
/replica-design

Usa estas capturas como referencia de la experiencia de una pregunta por
pantalla. Crea un diseño propio con transiciones suaves y que funcione bien en
celular.
```

**4** **Construir**

```
/replica-build

Construye primero el flujo completo: crear un formulario, publicarlo y recibir
una respuesta. Después la lógica condicional y al final el panel de respuestas.
```

**5** **Backend y pruebas**

```
/replica-backend

Agrega login, notificación por correo cuando llega una respuesta y un webhook
para mandar respuestas a n8n o Zapier. Después corre /replica-test.
```

**6** **Medir qué tan igual quedó**

```
/replica-diff

Compara mi clon contra la original y dime qué funciones obligatorias faltan para
poder lanzarlo.
```

**7** **Diferenciarte y lanzar**

```
/replica-entrepreneur > /replica-brand > /replica-launch > /replica-deploy

En entrepreneur: busca quejas sobre límites de respuestas y precios.
En launch: plan con respuestas ilimitadas y precio fijo.
```

**TIP PARA ESTE CASO**

Si trabajas con agencias o automatizaciones, el webhook nativo hacia n8n es un buen argumento de venta por sí solo.

CASO DE USO 04

# 04 CRM ligero

Tipo Pipedrive

| DIFICULTAD | IDEAL PARA | APP DE REFERENCIA |
|---|---|---|
| **Media** | **Pymes y equipos de ventas** | **Pipedrive** |

## Por qué conviene

Muchas pymes solo usan una fracción de un CRM grande y aun así pagan por usuario cada mes. Un CRM simple con pipeline visual, contactos y recordatorios cubre lo que la mayoría necesita.

## Qué vas a construir

- Pipeline visual de ventas con etapas arrastrables
- Contactos y empresas con historial
- Tareas y recordatorios de seguimiento
- Reporte simple de ventas por etapa

## Paso 0: abre tu proyecto en Claude Code

Desde tu terminal crea una carpeta para el clon y abre Claude Code dentro:

```
mkdir clon-crm && cd clon-crm
claude
```

## Prompts paso a paso

Copia cada prompt en Claude Code en este orden. Espera a que termine cada paso antes del siguiente.

### 1 Analizar la app

```
/replica-recon https://www.pipedrive.com

Analiza este CRM. Enfócate en el pipeline de ventas, la ficha de contacto, las tareas de seguimiento y los reportes básicos. Marca como fuera de alcance las funciones de IA, el marketplace y las herramientas de email marketing.
```

### 2 Planear la arquitectura

```
/replica-architect

Next.js y Postgres. El sistema debe soportar varios usuarios por empresa con permisos básicos (administrador y vendedor).
```

### 3 Diseño

```
/replica-design

Usa estas capturas como referencia del tablero de pipeline y la ficha de
contacto. Crea un diseño propio, sobrio y fácil de leer.
```

### 4 Construir

```
/replica-build

Construye primero el pipeline con tarjetas arrastrables entre etapas y la ficha
de contacto. Después las tareas y al final los reportes.
```

### 5 Backend y pruebas

```
/replica-backend

Agrega login, invitación de usuarios al equipo, permisos por rol y recordatorios
por correo de tareas pendientes. Después corre /replica-test.
```

### 6 Medir qué tan igual quedó

```
/replica-diff

Compara mi clon contra la original y dame el puntaje y la lista de funciones
obligatorias que faltan.
```

### 7 Diferenciarte y lanzar

```
/replica-entrepreneur > /replica-brand > /replica-launch > /replica-deploy

En entrepreneur: busca quejas sobre precio por usuario y complejidad.
En launch: precio fijo por empresa, no por usuario.
```

**TIP PARA ESTE CASO**

Puedes empezar usándolo en tu propio negocio para llevar tus clientes y después venderlo a negocios con el mismo problema.